

Pitch Estimation Network

Student: Zaicanu Raluca-Maria

TM, Year I

Table of Contents

Abstract	3
Introduction.....	3
State of the art	4
Theory	5
Implementation	6
Structure and reproducibility	6
Data ingestion and pairing	6
Audio preprocessing and framing	7
Reference processing and target construction.....	7
Network architecture.....	8
Training	9
Evaluation notebook and metrics.....	9
Experimental Results	10
Conclusion	12

Abstract

This report documents the design and implementation of a neural pitch estimation system trained to mimic a PiENet style architecture and evaluated on PTDB TUG. The system frames microphone speech into 32 ms windows with a 10 ms hop and predicts a log frequency pitch class for each frame, including an explicit unvoiced class. A WaveNet inspired stack of gated dilated convolutions provides temporal context while keeping the model lightweight. The implementation is divided into a training notebook that performs speaker disjoint splitting, augmentation, optimization, and checkpointing, and an evaluation notebook that reloads the checkpoint and reproduces metrics and pitch contour plots without retraining.

Introduction

Fundamental frequency estimation, often referred to as pitch tracking, is a central component of speech analysis and speech technology. It supports tasks such as prosody modelling, speaker characterization, voice conversion, and pitch synchronous analysis. Traditional signal processing algorithms remain widely used because they are interpretable and computationally efficient. However, their accuracy can deteriorate under noise, channel variability, and rapid voicing transitions.

Motivated by recent neural pitch estimators reviewed before, this project implements a compact pitch estimation network inspired by PiENet. The system predicts, at each analysis frame, a posterior distribution over a log spaced set of pitch bins together with an explicit unvoiced class.

Experiments use PTDB TUG, which provides microphone recordings and reference pitch trajectories derived from laryngograph measurements. The implementation is organized into a training notebook that performs data preparation, speaker disjoint splitting, augmentation, optimization, and checkpointing, and an evaluation notebook that reloads the saved checkpoint and reproduces metrics and pitch contour plots without retraining.

State of the art

The state of the art in pitch tracking spans classical periodicity based algorithms and modern learning based models. Time domain approaches such as YIN reduce octave errors through a normalized difference function and a principled selection rule, while dynamic programming-based methods such as RAPT emphasize temporal smoothness and robustness in practical speech recordings [1] [2].

Frequency domain and hybrid methods exploit harmonic structure and remain strong baselines in speech processing. In particular, the WORLD family popularized efficient F0 extraction via DIO, which is frequently used in speech synthesis pipelines and as reference baselines for model evaluation [3]. Probabilistic temporal models, exemplified by pYIN, combine local candidates with sequence constraints to improve consistency in difficult regions [4].

Neural estimators learn pitch-relevant representations directly from data and can be trained to be robust across speakers and recording conditions. CREPE demonstrated accurate pitch classification from waveform frames using a convolutional network that outputs a vector as a probability-like curve over pitch, showing what pitches are plausible for that frame[5]. Moreover, self-supervised learning has enabled pitch representations that reduce dependency on labeled trajectories, as illustrated by SPICE [6].

PiENet introduced a WaveNet-style dilated residual architecture for noise-robust fundamental frequency tracking and highlighted the importance of augmentation for channel and noise variability[7]. Subsequent work has explored compact designs aimed at real-time constraints, including hybrid approaches that combine signal processing front ends with lightweight neural components [8]. Joint modelling of voicing and pitch remains important because voicing errors often dominate perceptual failures, and robust speech studies provide useful guidance for evaluation protocols [9].

In summary, prior work motivates three design decisions adopted in this report. First, log spaced pitch bins reflect perceptual scaling and align naturally with cents-based evaluation. Second, temporal context should be modelled explicitly to reduce frame-level instability. Third, augmentation is essential when training on microphone speech where gain and noise conditions vary across speakers and sessions.

Theory

In voiced speech, vocal fold vibration produces a quasi periodic excitation that yields a fundamental period and therefore a fundamental frequency F_0 . Because many segments are unvoiced, pitch tracking combines two coupled problems: detecting whether a frame is voiced and estimating the frequency when voicing is present. Therefore, evaluation often separates voicing decision errors from frequency errors computed on voiced frames [10].

Frame-based analysis is commonly used to make the estimation problem well-posed. A short time window is assumed to be approximately stationary, allowing periodic structure to be detected within that window. PTDB TUG reference trajectories are provided at a 10 ms frame rate using a 32 ms analysis window. Therefore, matching the network framing to these settings simplifies alignment and avoids systematic timing offsets.

Neural pitch estimation can be formulated as regression or classification. Regression predicts a continuous frequency value, whereas classification predicts a discrete bin index. Classification is attractive because cross entropy provides a stable objective, posterior sharpness encodes uncertainty, and an explicit unvoiced class can be included. Moreover, a logarithmic bin spacing yields approximately constant resolution in musical cents across frequency.

Temporal context is crucial because pitch contours evolve smoothly and short frames may be ambiguous near voicing boundaries or in low energy regions. Dilated convolutions enlarge the receptive field without a proportional increase in parameters, and residual connections help stabilize deep stacks. Therefore, WaveNet inspired gated residual blocks provide an efficient mechanism for modelling pitch continuity while remaining suitable for low latency deployment.[7]

Implementation

Structure and reproducibility

The modelling choices that were touched upon in the State of the Art chapter were implemented, and this part explains how they are translated into a runnable system. The project is implemented in two Colab notebooks with clearly separated responsibilities. The first notebook trains the network from paired microphone waveforms and reference trajectories, while the second notebook is designed for rapid reuse and performs evaluation by loading a saved checkpoint from Google Drive. Reproducibility is supported by storing the full preprocessing configuration inside the checkpoint. In addition to network weights, the checkpoint stores the bin centers, target sample rate, window length, hop length, and binning range. Therefore, the evaluation notebook can exactly reconstruct the processing pipeline used during training, which reduces the risk of silent mismatches between experiments.

Data ingestion and pairing

PTDB TUG organizes speech recordings by gender and signal category, providing microphone signals, laryngograph signals, and reference trajectories. The implementation focuses on microphone input because it matches the practical scenario of acoustic pitch estimation. For each waveform, the corresponding reference file is located by a deterministic filename mapping that replaces the microphone prefix with the reference prefix and changes the extension from wav to f0. This pairing yields a list of waveform and reference path tuples that form the basis for both training and evaluation[11].

Speaker ID	Age	Sex	Home Country	Sentences			Comment
F01	40	Female	Ireland	sa1,2	sx3-47	si453-641	Error: si1912,1913
F02	25	Female	USA	sa1,2	sx48-92	si642-830	
F03	22	Female	Canada	sa1,2	sx93-137	si831-1019	
F04	26	Female	Canada	sa1,2	sx138-182	si1020-1208	
F05	48	Female	USA	sa1,2	sx183-227	si1209-1397	
F06	28	Female	USA	sa1,2	sx228-272	si1398-1586	
F07	24	Female	USA	sa1,2	sx273-317	si1587-1775	
F08	22	Female	England	sa1,2	sx318-362	si1776-1964	
F09	22	Female	USA	sa1,2	sx363-407	si1965-2153	
F10	35	Female	USA	sa1,2	sx408-452	si2154-2342	
M01	24	Male	South Africa	sa1,2	sx3-47	si453-641	
M02	40	Male	England	sa1,2	sx48-92	si642-830	
M03	35	Male	England	sa1,2	sx93-137	si831-1019	
M04	26	Male	USA	sa1,2	sx138-182	si1020-1208	
M05	25	Male	England	sa1,2	sx183-227	si1209-1397	
M06	23	Male	USA	sa1,2	sx228-272	si1398-1586	
M07	24	Male	USA	sa1,2	sx273-317	si1587-1775	
M08	24	Male	England	sa1,2	sx318-362	si1776-1964	
M09	24	Male	Canada	sa1,2	sx363-407	si1965-2153	
M10	33	Male	USA	sa1,2	sx408-452	si2154-2342	

Figure 1 Speaker profiles

Although utterance-level random splits are common, they can leak speaker characteristics across partitions and lead to overly optimistic results. Therefore, the notebook constructs a speaker to utterances mapping by parsing speaker identifiers from filenames and then assigns speakers to training, validation, and test partitions. A fixed random seed is used during shuffling so that the same speaker disjoint split can be recreated. In the recorded run, the split yields 3774 training utterances, 472 validation utterances, and 472 test utterances across 20 speakers.

Audio preprocessing and framing

Audio is loaded as float32 and converted to mono by channel averaging when necessary. The original recordings are provided at a higher sampling rate, whereas the model operates at 16 kHz. Therefore, each waveform is resampled using polyphase filtering with integer up and down factors derived from the greatest common divisor of the source and target rates. After resampling, peak normalization reduces sensitivity to recording gain and simplifies the behavior of waveform level augmentation.

The framing strategy mirrors the reference extraction settings. A 512 sample window at 16 kHz corresponds to 32 ms, and a 160 sample hop corresponds to 10 ms. Therefore, the network produces one prediction per reference frame without requiring interpolation. Frames are generated with symmetric padding of half a window at both ends, and the number of frames is computed as the ceiling of the original length divided by the hop.

```
# Framing (10 ms hop at 16 kHz -> 160 samples)
TARGET_SR = 16000
WINLEN = 512
HOP = 160
HOP_SEC = HOP / TARGET_SR
```

Figure 2 Framing parameters

Reference processing and target construction

Following the classification approach described in Theory, the continuous frequency axis is discretized into 351 bins. The first bin represents the unvoiced state, while the remaining bins are log-spaced between 50 Hz and 500 Hz. Each voiced reference value is mapped to the nearest voiced bin by minimizing log frequency distance. Therefore, the learning target is stable even when

the reference frequency moves slightly within a narrow band, and cross entropy remains well conditioned.

Network architecture

The implemented model, named PiENetLike, consumes framed audio with shape B by T by 512 and outputs logits with shape B by T by 351, where each logit vector corresponds to a pitch bin. The first stage is a one-by-one convolution that projects the 512 sample frame dimension into 128 residual channels. Dropout with probability 0.4 is applied early to reduce overfitting and to encourage robustness to local waveform variations.

```
class PiENetLike(nn.Module):
    def __init__(self,
                 winlen=HTMLEN,
                 nbins=NBINS,
                 residual_channels=128,
                 filter_width=5,
                 dilations=(1,2,4,8,1,2,4,8),
                 postnet_channels=256,
                 dropout=0.4):
        super().__init__()
        self.in_proj = nn.Conv1d(winlen, residual_channels, kernel_size=1)
        self.dropout = nn.Dropout(p=dropout)

        self.fg_convs = nn.ModuleList([
            nn.Conv1d(residual_channels, 2*residual_channels,
                      kernel_size=filter_width, dilation=d,
                      padding=(filter_width//2)*d)
            for d in dilations
        ])
        self.skip_convs = nn.ModuleList([nn.Conv1d(residual_channels, residual_channels, kernel_size=1) for _ in dilations])
        self.out_convs = nn.ModuleList([nn.Conv1d(residual_channels, residual_channels, kernel_size=1) for _ in dilations])

        self.post1 = nn.Conv1d(residual_channels, postnet_channels,
                                kernel_size=filter_width, padding=filter_width//2)
        self.post2 = nn.Conv1d(postnet_channels, nbins,
                                kernel_size=filter_width, padding=filter_width//2)

    def forward(self, frames_bt):
        # frames_bt: [B,T,512]
        x = frames_bt.transpose(1, 2) # [B,512,T]
        x = self.dropout(x)
        r = torch.tanh(self.in_proj(x)) # [B,R,T]

        skips = []
        h = r
        for fg, sk, out in zip(self.fg_convs, self.skip_convs, self.out_convs):
            y = fg(h) # [B,2R,T]
            a, b = y.chunk(2, dim=1)
            y = torch.tanh(a) * torch.sigmoid(b)
            skips.append(sk(y)) # [B,R,T]
            h = out(y) + h # residual

        s = torch.stack(skips, dim=0).sum(dim=0) # [B,R,T]
        z = torch.relu(self.post1(s))
        logits = self.post2(z) # [B,NBINS,T]
        return logits.transpose(1, 2) # [B,T,NBINS]

model = PiENetLike().to(device)
```

Figure 3 PiENet-like Class

Temporal modelling is performed by eight gated dilated convolution blocks with kernel width 5 and dilations 1, 2, 4, and 8 repeated twice. Each block applies a convolution that outputs twice the residual width and then splits it into two halves. One half is passed through a tanh function, and the other through the sigmoid, and their elementwise product forms a gated activation. Moreover, each block contributes both a residual update and a skip contribution. Skip contributions are summed across blocks and passed to a small post network that maps 128 channels to 256 channels and then to the 351 pitch logits.

Training

Training uses mini-batches built from variable-length utterances. Because sequences have different numbers of frames, each batch is padded to the maximum sequence length in the batch. The target class tensor uses a dedicated ignore value for padded positions, and cross-entropy loss is computed only on valid frames. Therefore, optimization is not influenced by artificial padding.

Waveform level augmentation is applied to improve robustness, consistent with the State of the Art discussion of noise robust learning. Additive Gaussian noise is injected with a randomly sampled signal-to-noise ratio between 0 dB and 20 dB, and a random gain between 0.7 and 1.2 is applied with clipping to the valid amplitude range. These transformations encourage the model to focus on periodic structure rather than absolute amplitude.

Optimization uses Adam with a learning rate of $1e-4$ and weight decay of $1e-4$. Gradient norms are clipped to 3.0, which reduces the risk of unstable updates in early epochs. Training was intended for 40 epochs with batch size 8, but the best model was selected after epoch 20, based on validation loss. When an improved validation loss is observed, the notebook writes a checkpoint that stores both the model parameters and the full preprocessing configuration.

Evaluation notebook and metrics

The evaluation notebook loads the checkpoint and reconstructs bin centers and framing parameters before instantiating the model. Because the checkpoint contains numpy arrays, the loader enables full checkpoint loading in modern PyTorch versions. The same speaker disjoint split procedure is then rerun, ensuring that validation and test sets match the training protocol.

Metrics follow standard practice in the pitch estimation literature. A voicing decision error rate measures disagreement between predicted and reference voicing. On frames where both the reference and the prediction are voiced, the frequency error is measured in cents, which is consistent with the log frequency representation. Finally, an accuracy within 50 cents summarizes how often the system stays within a perceptually small tolerance. “Within 50 cents” means the estimate is within half a semitone of the true pitch, which is commonly treated as a small,

perceptually meaningful tolerance. In the Figure below, f_2 represents the predicted frequency, and f_1 the reference frequency.[12]

$$1200 \log_2 \left(\frac{f_2}{f_1} \right) \text{ cents}$$

Figure 4 cent formula

Experimental Results

The proposed PiENet like pitch estimator was evaluated on PTDB TUG microphone recordings using a speaker disjoint split, so that generalization is measured on unseen speakers rather than on utterances from the same speakers seen during training. On the validation set, the system achieved a voicing error rate of about 0.0544, a voiced only mean absolute error of about 18.96 cents, and an accuracy within 50 cents of about 0.9834. On the test set, the voicing error rate remained comparable at about 0.0542, while the voiced only mean absolute error decreased to about 13.58 cents, with an accuracy within 50 cents of about 0.9751. Therefore, the model stays within a perceptually small tolerance for the large majority of voiced frames, and the low cents error indicates that the classifier resolution and temporal context are sufficient to track typical intonation trajectories without frequent octave mistakes, which aligns with the State of the Art discussion on classification based neural estimators.

Qualitative inspection of predicted contours further supports these aggregate metrics. In the example utterances from speaker M03, the predicted trajectories closely overlap the reference tracks over most voiced regions, capturing both gradual declinations and short pitch movements with minimal lag. Moreover, the model tends to preserve smoothness across frames, which is expected from the dilated convolutional receptive field described earlier, and it reduces isolated frame outliers that are common in purely local estimators. The remaining discrepancies are concentrated around difficult regions, especially near voiced unvoiced boundaries and short or low energy voiced segments, where small voicing disagreements can occur and where the reference contour sometimes contains abrupt changes that are challenging to follow perfectly at 10 ms resolution. However, these deviations are typically small in magnitude and do not dominate the

perceptual error distribution, which is consistent with the high accuracy within 50 cents reported above.

```
*** eval: 100% [59/59 [05:06:00:00, 5.19s/it]
VAL metrics: {'VoicingErrorRate': 0.05437770612159018, 'MAE_cents_voiced': 18.958011061458265, 'Accuracy_within_50cents': 0.9834175220990585}
eval: 100% [59/59 [04:06:00:00, 4.17s/it]TEST metrics: {'VoicingErrorRate': 0.05424255853749529, 'MAE_cents_voiced': 13.58192342014636, 'Accuracy_within_50cents': 0.9758952427670107}
```

Figure 5 VoicingErrorRate and Accuracy

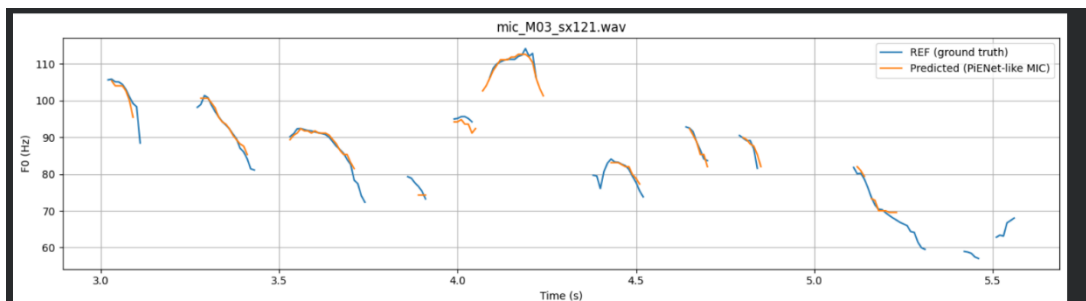


Figure 6 Visualization of Prediction vs REF sx121

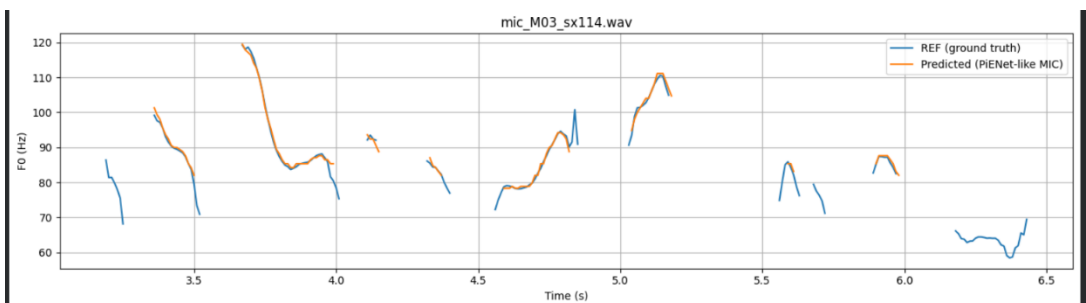


Figure 7 Visualisation of Prediction vs REF sx114

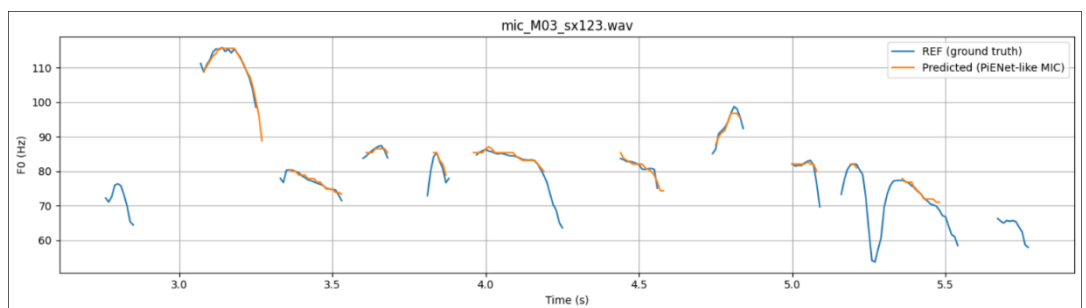


Figure 8 Visualization of Prediction vs REF sx123

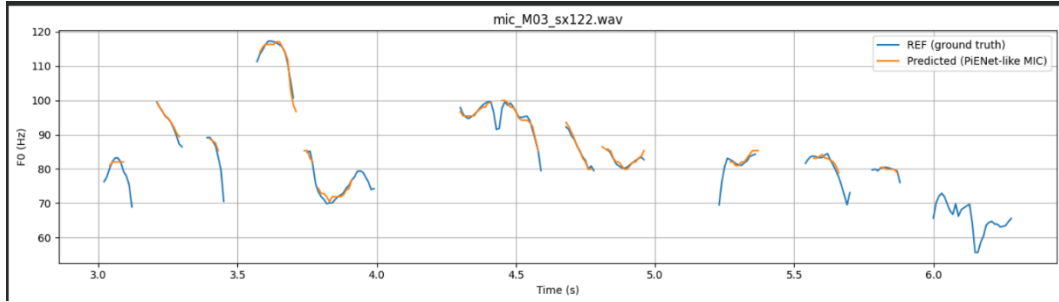


Figure 9 Visualization of Prefiction vs REF sx122

Conclusion

In this project, a PiNet-like neural network was trained to estimate pitch from microphone speech recordings in the PTDB TUG dataset. The model predicts pitch frame by frame using log-spaced pitch classes and a separate unvoiced class, so it learns both pitch tracking and voicing at the same time. The evaluation on unseen speakers shows strong performance: most voiced frames are predicted very close to the reference pitch, and the plotted examples confirm that the predicted pitch curves usually follow the ground truth well. Overall, the results suggest that the network is able to generalize across speakers and produce stable pitch trajectories from raw microphone audio.

There are still clear ways to improve the system. First, training could include more realistic distortions such as reverberation, different microphones, and background noise types, so the model becomes stronger in real world conditions. Second, the output could be smoothed over time using a simple decoding method, which would reduce small jumps and improve voiced to unvoiced transitions. Third, instead of choosing the single best pitch bin with argmax, the model could compute a weighted average of nearby bins to get a slightly more precise pitch estimate. Finally, comparing the model directly against standard baselines like RAPT, YIN, or pYIN on the same test set would make the conclusions stronger and easier to position relative to prior work.

References

- [1] A. de Cheveigné and H. Kawahara, “YIN, a fundamental frequency estimator for speech and music,” *J. Acoust. Soc. Am.*, vol. 111, no. 4, pp. 1917–1930, Apr. 2002, doi: 10.1121/1.1458024.
- [2] David Talkin, “A Robust Algorithm for Pitch Tracking”.
- [3] M. Morise, F. Yokomori, and K. Ozawa, “WORLD: A vocoder-based high-quality speech synthesis system for real-time applications,” *IEICE Trans. Inf. Syst.*, vol. E99D, no. 7, pp. 1877–1884, Jul. 2016, doi: 10.1587/transinf.2015EDP7457.
- [4] M. Mauch and S. Dixon, “PYIN: A FUNDAMENTAL FREQUENCY ESTIMATOR USING PROBABILISTIC THRESHOLD DISTRIBUTIONS.” [Online]. Available: <http://code.soundsoftware.ac.uk/projects/pyin>
- [5] J. W. Kim, J. Salamon, P. Li, and J. P. Bello, “CREPE: A CONVOLUTIONAL REPRESENTATION FOR PITCH ESTIMATION.” [Online]. Available: <https://github.com/marl/crepe>
- [6] B. Gfeller, C. Frank, D. Roblek, M. Sharifi, M. Tagliasacchi, and M. Velimirović, “SPICE: Self-supervised Pitch Estimation,” Sep. 2020, doi: 10.1109/TASLP.2020.2982285.
- [7] M. Airaksinen, L. Juvela, P. Alku, and O. Rasanen, “Data Augmentation Strategies for Neural Network F0 Estimation,” in *ICASSP, IEEE International Conference on Acoustics, Speech and Signal Processing - Proceedings*, Institute of Electrical and Electronics Engineers Inc., May 2019, pp. 6485–6489. doi: 10.1109/ICASSP.2019.8683041.
- [8] K. Subramani, J.-M. Valin, J. Bütche, P. Smaragdis, and M. Goodwin, “NOISE-ROBUST DSP-ASSISTED NEURAL PITCH ESTIMATION WITH VERY LOW COMPLEXITY.”
- [9] T. Drugman and A. Alwan, “Joint robust voicing detection and pitch estimation based on residual harmonics,” in *Proceedings of the Annual Conference of the International Speech Communication Association, INTERSPEECH*, 2011, pp. 1973–1976. doi: 10.21437/interspeech.2011-519.

- [10] M. Morrison, C. Hsieh, N. Pruyne, and B. Pardo, “Cross-domain Neural Pitch and Periodicity Estimation,” Aug. 2024, [Online]. Available: <http://arxiv.org/abs/2301.12258>
- [11] G. Pirker, M. Wohlmayr, S. Petrik, and F. Pernkopf, “The Pitch-Tracking Database from Graz University of Technology,” 2012.
- [12] C. P. Dettmann and L. Taylor-West, “Algebraic tunings,” *Journal of Mathematics and Music*, vol. 18, no. 2, pp. 203–216, 2024, doi: 10.1080/17459737.2023.2234126.